

```
1 using System;
2 using System.Drawing;
3 using System.Windows.Forms;
4
5 namespace RogueOfTheDarkAges
6 {
7     public class MainForm : Form
8     {
9         private Random random;
10        private PlayerClass player;
11        private Enemy currentEnemy;
12        private int currentLevel;
13        private bool bossBattle;
14        private bool waitingForUpgrade;
15        private bool gameOver;
16
17        private string[] environments;
18        private string[] enemyNames;
19
20        private Label titleLabel;
21        private Label setupHeaderLabel;
22        private Label playerHeaderLabel;
23        private Label enemyHeaderLabel;
24        private Label actionsHeaderLabel;
25        private Label upgradeHeaderLabel;
26        private Label logHeaderLabel;
27        private Label statusLabel;
28
29        private Label classSelectLabel;
30        private Label levelLabel;
31        private Label classLabel;
32        private Label playerHealthLabel;
33        private Label weaponLabel;
34        private Label bonusDamageLabel;
35        private Label potionLabel;
36        private Label enemyNameLabel;
37        private Label enemyHealthLabel;
38        private Label environmentLabel;
39
40        private ProgressBar playerHealthBar;
41        private ProgressBar enemyHealthBar;
42
43        private ComboBox classSelector;
44        private Button startButton;
45        private Button restartButton;
46        private Button attackButton;
47        private Button blockButton;
48        private Button healButton;
49        private Button upgradeHealthButton;
```

```
50     private Button upgradeDamageButton;
51     private Button upgradePotionButton;
52
53     private RichTextBox logBox;
54
55     public MainForm()
56     {
57         random = new Random();
58
59         environments = new string[]
60         {
61             "a fog-covered swamp where the earth squelches beneath your boots",
62             "the ruins of an old watchtower scattered with broken stone",
63             "a pine forest clearing lit by a pale morning sun",
64             "a muddy battlefield abandoned after a brutal clash",
65             "a narrow mountain pass with cold wind cutting through the rocks",
66             "an overgrown monastery courtyard filled with creeping ivy",
67             "a torchlit castle hallway smelling of smoke and iron",
68             "a river crossing where the current crashes against worn pillars",
69             "a village square left eerily silent after nightfall",
70             "a windswept cliffside where gulls cry over the sea"
71         };
72
73         enemyNames = new string[]
74         {
75             "Brigand",
76             "Mercenary",
77             "Raider",
78             "Footman",
79             "Pirate",
80             "Hunter",
81             "Knight",
82             "Marauder",
83             "Bandit Captain",
84             "Grave Robber",
85             "Soldier",
86             "Guard",
87             "Scout",
88             "Warrior"
89         };
90
91         BuildInterface();
92         ResetToMenu();
93     }
```

```
94
95     private void BuildInterface()
96     {
97         Text = "Rogue of the Dark Ages";
98         StartPosition = FormStartPosition.CenterScreen;
99         ClientSize = new Size(1120, 820);
100        FormBorderStyle = FormBorderStyle.FixedSingle;
101        MaximizeBox = false;
102        BackColor = Color.FromArgb(20, 22, 28);
103        ForeColor = Color.White;
104        Font = new Font("Segoe UI", 10F, FontStyle.Regular);
105
106        titleLabel = new Label();
107        titleLabel.Text = "ROGUE OF THE DARK AGES";
108        titleLabel.Font = new Font("Segoe UI", 22F, FontStyle.Bold);
109        titleLabel.ForeColor = Color.Gold;
110        titleLabel.AutoSize = true;
111        titleLabel.Location = new Point(24, 18);
112        Controls.Add(titleLabel);
113
114        statusLabel = new Label();
115        statusLabel.Font = new Font("Segoe UI", 10F, FontStyle.Bold);
116        statusLabel.ForeColor = Color.LightSkyBlue;
117        statusLabel.AutoSize = false;
118        statusLabel.Location = new Point(28, 58);
119        statusLabel.Size = new Size(1040, 24);
120        Controls.Add(statusLabel);
121
122        setupHeaderLabel = CreateSectionLabel("Game Setup", 28, 95);
123        Controls.Add(setupHeaderLabel);
124
125        classSelectLabel = new Label();
126        classSelectLabel.Text = "Choose Class:";
127        classSelectLabel.Location = new Point(30, 128);
128        classSelectLabel.Size = new Size(100, 24);
129        classSelectLabel.ForeColor = Color.WhiteSmoke;
130        Controls.Add(classSelectLabel);
131
132        classSelector = new ComboBox();
133        classSelector.DropDownStyle = ComboBoxStyle.DropDownList;
134        classSelector.Location = new Point(135, 125);
135        classSelector.Size = new Size(180, 28);
136        classSelector.Items.Add("Knight");
137        classSelector.Items.Add("Vanguard");
138        classSelector.Items.Add("Hunter");
139        classSelector.Items.Add("Rogue");
140        classSelector.Items.Add("Mage");
141        classSelector.SelectedIndex = 0;
142        Controls.Add(classSelector);
```

```
143
144         startButton = CreateButton("Start Game", 330, 123, 120, 34);
145         startButton.Click += StartButton_Click;
146         Controls.Add(startButton);
147
148         restartButton = CreateButton("Restart", 460, 123, 100, 34);
149         restartButton.Click += RestartButton_Click;
150         Controls.Add(restartButton);
151
152         playerHeaderLabel = CreateSectionLabel("Player", 28, 180);
153         Controls.Add(playerHeaderLabel);
154
155         levelLabel = CreateInfoLabel(30, 214, 460);
156         Controls.Add(levelLabel);
157
158         classLabel = CreateInfoLabel(30, 244, 460);
159         Controls.Add(classLabel);
160
161         playerHealthLabel = CreateInfoLabel(30, 274, 460);
162         Controls.Add(playerHealthLabel);
163
164         playerHealthBar = new ProgressBar();
165         playerHealthBar.Location = new Point(30, 304);
166         playerHealthBar.Size = new Size(300, 18);
167         playerHealthBar.Style = ProgressBarStyle.Continuous;
168         Controls.Add(playerHealthBar);
169
170         weaponLabel = CreateInfoLabel(30, 334, 460);
171         Controls.Add(weaponLabel);
172
173         bonusDamageLabel = CreateInfoLabel(30, 364, 460);
174         Controls.Add(bonusDamageLabel);
175
176         potionLabel = CreateInfoLabel(30, 394, 460);
177         Controls.Add(potionLabel);
178
179         enemyHeaderLabel = CreateSectionLabel("Enemy", 540, 180);
180         Controls.Add(enemyHeaderLabel);
181
182         enemyNameLabel = CreateInfoLabel(542, 214, 540);
183         Controls.Add(enemyNameLabel);
184
185         enemyHealthLabel = CreateInfoLabel(542, 244, 540);
186         Controls.Add(enemyHealthLabel);
187
188         enemyHealthBar = new ProgressBar();
189         enemyHealthBar.Location = new Point(542, 274);
190         enemyHealthBar.Size = new Size(300, 18);
191         enemyHealthBar.Style = ProgressBarStyle.Continuous;
```

```
192         Controls.Add(enemyHealthBar);
193
194         environmentLabel = new Label();
195         environmentLabel.Location = new Point(542, 304);
196         environmentLabel.Size = new Size(540, 90);
197         environmentLabel.ForeColor = Color.LightCyan;
198         environmentLabel.AutoSize = false;
199         Controls.Add(environmentLabel);
200
201         actionsHeaderLabel = CreateSectionLabel("Actions", 28, 450);
202         Controls.Add(actionsHeaderLabel);
203
204         attackButton = CreateButton("Attack", 30, 485, 130, 42);
205         attackButton.Click += AttackButton_Click;
206         Controls.Add(attackButton);
207
208         blockButton = CreateButton("Block", 180, 485, 130, 42);
209         blockButton.Click += BlockButton_Click;
210         Controls.Add(blockButton);
211
212         healButton = CreateButton("Heal", 330, 485, 130, 42);
213         healButton.Click += HealButton_Click;
214         Controls.Add(healButton);
215
216         upgradeHeaderLabel = CreateSectionLabel("Upgrade Choice", 28, 560);
217         Controls.Add(upgradeHeaderLabel);
218
219         upgradeHealthButton = CreateButton("+10 Max Health", 30, 595, 150, 42);
220         upgradeHealthButton.Click += UpgradeHealthButton_Click;
221         Controls.Add(upgradeHealthButton);
222
223         upgradeDamageButton = CreateButton("+2 Damage", 190, 595, 130, 42);
224         upgradeDamageButton.Click += UpgradeDamageButton_Click;
225         Controls.Add(upgradeDamageButton);
226
227         upgradePotionButton = CreateButton("+1 Potion", 330, 595, 130, 42);
228         upgradePotionButton.Click += UpgradePotionButton_Click;
229         Controls.Add(upgradePotionButton);
230
231         logHeaderLabel = CreateSectionLabel("Battle Log", 560, 430);
232         Controls.Add(logHeaderLabel);
233
234         logBox = new RichTextBox();
235         logBox.Location = new Point(560, 470);
236         logBox.Size = new Size(520, 315);
```

```
237         logBox.ReadOnly = true;
238         logBox.Font = new Font("Consolas", 10F, FontStyle.Regular);
239         logBox.BackColor = Color.FromArgb(12, 14, 18);
240         logBox.ForeColor = Color.WhiteSmoke;
241         logBox.BorderStyle = BorderStyle.FixedSingle;
242         logBox.ScrollBars = RichTextBoxScrollBars.Vertical;
243         logBox.DetectUrls = false;
244         Controls.Add(logBox);
245     }
246
247     private Label CreateSectionLabel(string text, int x, int y)
248     {
249         Label label = new Label();
250         label.Text = text;
251         label.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
252         label.ForeColor = Color.Goldenrod;
253         label.Location = new Point(x, y);
254         label.Size = new Size(280, 24);
255         return label;
256     }
257
258     private Label CreateInfoLabel(int x, int y, int width)
259     {
260         Label label = new Label();
261         label.Location = new Point(x, y);
262         label.Size = new Size(width, 24);
263         label.ForeColor = Color.WhiteSmoke;
264         return label;
265     }
266
267     private Button CreateButton(string text, int x, int y, int width, int height)
268     {
269         Button button = new Button();
270         button.Text = text;
271         button.Location = new Point(x, y);
272         button.Size = new Size(width, height);
273         button.FlatStyle = FlatStyle.Flat;
274         button.FlatAppearance.BorderSize = 1;
275         button.FlatAppearance.BorderColor = Color.FromArgb(80, 90, 110);
276         button.BackColor = Color.FromArgb(38, 43, 54);
277         button.ForeColor = Color.White;
278         button.Cursor = Cursors.Hand;
279         return button;
280     }
281
282     private void ResetToMenu()
283     {
```

```
284         player = null;
285         currentEnemy = null;
286         currentLevel = 0;
287         bossBattle = false;
288         waitingForUpgrade = false;
289         gameOver = false;
290
291         levelLabel.Text = "Level: -";
292         classLabel.Text = "Class: -";
293         playerHealthLabel.Text = "Health: -";
294         weaponLabel.Text = "Weapon: -";
295         bonusDamageLabel.Text = "Bonus Damage: -";
296         potionLabel.Text = "Potions: -";
297
298         enemyNameLabel.Text = "Enemy: -";
299         enemyHealthLabel.Text = "Enemy Health: -";
300         environmentLabel.Text = "Environment: Choose a class and
            start a new run.";
301
302         playerHealthBar.Value = 0;
303         enemyHealthBar.Value = 0;
304
305         logBox.Clear();
306         AddHeaderLog("Welcome to Rogue of the Dark Ages.");
307         AddInfoLog("Block reduces incoming damage to 25% and has a
            25% chance to counter.");
308         AddInfoLog("Choose a class, then press Start Game.");
309
310         statusLabel.Text = "Ready to begin.";
311         SetBattleButtons(false);
312         SetUpgradeButtons(false);
313
314         startButton.Enabled = true;
315         classSelector.Enabled = true;
316     }
317
318     private void StartButton_Click(object sender, EventArgs e)
319     {
320         player = CreateSelectedPlayerClass();
321         player.Potions = 4;
322         player.BonusDamage = 0;
323         currentLevel = 1;
324         bossBattle = false;
325         waitingForUpgrade = false;
326         gameOver = false;
327
328         logBox.Clear();
329         AddPlayerLog("You chose the " + player.ClassName + ".");
330         AddInfoLog("Starting weapon: " + player.Weapon.Name);
```

```
331         AddInfoLog(player.GetClassDescription());
332         AddInfoLog("You start with 4 potions.");
333
334         startButton.Enabled = false;
335         classSelector.Enabled = false;
336
337         StartCurrentLevel();
338     }
339
340     private void RestartButton_Click(object sender, EventArgs e)
341     {
342         ResetToMenu();
343     }
344
345     private PlayerClass CreateSelectedPlayerClass()
346     {
347         string selectedClass = classSelector.SelectedItem.ToString();
348
349         if (selectedClass == "Knight")
350         {
351             return new Knight();
352         }
353         else if (selectedClass == "Vanguard")
354         {
355             return new Vanguard();
356         }
357         else if (selectedClass == "Hunter")
358         {
359             return new Hunter();
360         }
361         else if (selectedClass == "Rogue")
362         {
363             return new Rogue();
364         }
365         else
366         {
367             return new Mage();
368         }
369     }
370
371     private void StartCurrentLevel()
372     {
373         waitingForUpgrade = false;
374         SetUpgradeButtons(false);
375         SetBattleButtons(true);
376
377         player.RestoreHealthToFull();
378
379         if (currentLevel <= 6)
```



```
380     {
381         bossBattle = false;
382         currentEnemy = CreateEnemy(currentLevel);
383         string place = environments[random.Next           ↗
            (environments.Length)];
384
385         AddLog("");
386         AddHeaderLog("=== Level " + currentLevel + " ===");
387         AddInfoLog("You arrive in " + place + ".");
388         AddEnemyLog("A " + currentEnemy.Name + " " +      ↗
            currentEnemy.EnemyClassName + " appears with a " + ↗
            currentEnemy.Weapon.Name + ".");
389         environmentLabel.Text = "Environment: " + place;
390         statusLabel.Text = "Choose an action: Attack, Block, or ↗
            Heal.";
391     }
392     else
393     {
394         bossBattle = true;
395         currentEnemy = new Enemy("Ancient Dragon", "Boss", 95, ↗
            new Weapon(WeaponType.IgnitedClaws, "Ignited Claws", ↗
            10, 15));
396         currentEnemy.IsBoss = true;
397         currentEnemy.BonusDamage = 4;
398         currentEnemy.Potions = 1;
399
400         AddLog("");
401         AddHeaderLog("=== Final Boss ===");
402         AddDefeatLog("An Ancient Dragon flies down from the sky ↗
            in a storm of fire and ash.");
403         environmentLabel.Text = "Environment: a storm of fire and ↗
            ash";
404         statusLabel.Text = "Final Boss battle. Choose your action ↗
            carefully.";
405     }
406
407     UpdateUi();
408 }
409
410 private void AttackButton_Click(object sender, EventArgs e)
411 {
412     TakeTurn(BattleAction.Attack);
413 }
414
415 private void BlockButton_Click(object sender, EventArgs e)
416 {
417     TakeTurn(BattleAction.Block);
418 }
419
```

```
420     private void HealButton_Click(object sender, EventArgs e)
421     {
422         TakeTurn(BattleAction.Heal);
423     }
424
425     private void TakeTurn(BattleAction playerAction)
426     {
427         if (player == null || currentEnemy == null || gameOver ||
            waitingForUpgrade)
428         {
429             return;
430         }
431
432         AddLog("");
433         AddHeaderLog(bossBattle ? "=== Boss Battle ===" : "=== Battle
            ===");
434
435         BattleAction enemyAction = GetEnemyAction(currentEnemy,
            bossBattle);
436
437         bool playerBlocked = playerAction == BattleAction.Block;
438         bool enemyBlocked = enemyAction == BattleAction.Block;
439
440         if (playerAction == BattleAction.Heal)
441         {
442             if (player.Potions > 0)
443             {
444                 int healAmount = player.UsePotion(random);
445                 AddPlayerLog("You used a potion and healed " +
            healAmount + " health.");
446             }
447             else
448             {
449                 AddWarningLog("You do not have any potions.");
450             }
451         }
452         else if (playerAction == BattleAction.Attack)
453         {
454             DoAttack(player, currentEnemy, enemyBlocked, true);
455         }
456         else
457         {
458             AddInfoLog("You brace for impact and prepare to block.");
459         }
460
461         if (!currentEnemy.IsAlive())
462         {
463             EndBattle(true);
464             return;

```

```
465     }
466
467     if (enemyAction == BattleAction.Heal)
468     {
469         int healAmount = currentEnemy.UsePotion(random);
470         AddEnemyLog("The " + currentEnemy.Name + " used a potion  ↗
            and healed " + healAmount + " health.");
471     }
472     else if (enemyAction == BattleAction.Attack)
473     {
474         DoAttack(currentEnemy, player, playerBlocked, false);
475     }
476     else
477     {
478         AddEnemyLog("The " + currentEnemy.Name + " prepares to  ↗
            block.");
479     }
480
481     if (!player.IsAlive())
482     {
483         EndBattle(false);
484         return;
485     }
486
487     statusLabel.Text = "Choose your next action.";
488     UpdateUi();
489 }
490
491 private void DoAttack(Character attacker, Character defender,  ↗
    bool defenderBlocked, bool playerIsAttacking)
492 {
493     int damage = attacker.Attack(random);
494     bool critical = random.Next(1, 101) <=  ↗
        attacker.GetCriticalChance();
495
496     if (critical)
497     {
498         damage += random.Next(3, 7);
499     }
500
501     int originalDamage = damage;
502     bool countered = false;
503     int counterChance = 25;
504
505     if (defenderBlocked)
506     {
507         damage = (int)Math.Ceiling(originalDamage * 0.25);
508
509         if (damage < 1)
```

```

510         {
511             damage = 1;
512         }
513
514         if (random.Next(1, 101) <= counterChance)
515         {
516             countered = true;
517         }
518     }
519
520     defender.Health -= damage;
521
522     if (defender.Health < 0)
523     {
524         defender.Health = 0;
525     }
526
527     string attackerName;
528     string targetName;
529
530     if (playerIsAttacking)
531     {
532         attackerName = "Player";
533         targetName = defender.Name;
534     }
535     else
536     {
537         attackerName = "The " + attacker.Name;
538         targetName = "player";
539     }
540
541     if (defenderBlocked)
542     {
543         if (critical)
544         {
545             if (playerIsAttacking)
546             {
547                 AddPlayerLog(attackerName + " lands a critical hit with " + attacker.Weapon.Name + " for " + originalDamage + " damage, but it was blocked down to " + damage + ".");
548             }
549             else
550             {
551                 AddEnemyLog(attackerName + " lands a critical hit with " + attacker.Weapon.Name + " for " + originalDamage + " damage, but it was blocked down to " +
552

```

```

        damage + ".");
554     }
555 }
556 else
557 {
558     if (playerIsAttacking)
559     {
560         AddPlayerLog(attackerName + " attacks " +
        targetName + " with " + attacker.Weapon.Name +
561         " for " + originalDamage + " damage,
        but it was blocked down to " +
        damage + ".");
562     }
563     else
564     {
565         AddEnemyLog(attackerName + " attacks " +
        targetName + " with " + attacker.Weapon.Name +
566         " for " + originalDamage + " damage,
        but it was blocked down to " +
        damage + ".");
567     }
568 }
569 }
570 else
571 {
572     if (critical)
573     {
574         if (playerIsAttacking)
575         {
576             AddPlayerLog(attackerName + " lands a critical
        hit with " + attacker.Weapon.Name +
577             " and deals " + damage + " damage.");
578         }
579         else
580         {
581             AddEnemyLog(attackerName + " lands a critical hit
        with " + attacker.Weapon.Name +
582             " and deals " + damage + " damage.");
583         }
584     }
585     else
586     {
587         if (playerIsAttacking)
588         {
589             AddPlayerLog(attackerName + " attacks " +
        targetName + " with " + attacker.Weapon.Name +
590             " and deals " + damage + " damage.");
591         }
592         else

```

```
593         {
594             AddEnemyLog(attackerName + " attacks " +
595                         targetName + " with " + attacker.Weapon.Name +
596                         " and deals " + damage + " damage.");
597         }
598     }
599
600     if (countered && defender.IsAlive())
601     {
602         attacker.Health -= originalDamage;
603
604         if (attacker.Health < 0)
605         {
606             attacker.Health = 0;
607         }
608
609         string counterAttackerName;
610         string counterTargetName;
611
612         if (playerIsAttacking)
613         {
614             counterAttackerName = "The " + defender.Name;
615             counterTargetName = "you";
616             AddWarningLog(counterAttackerName + " counters and
617                         deals " + originalDamage + " damage to " +
618                         counterTargetName + "!");
619         }
620         else
621         {
622             counterAttackerName = "You";
623             counterTargetName = attacker.Name;
624             AddVictoryLog(counterAttackerName + " counterattacks
625                         and deals " + originalDamage + " damage to " +
626                         counterTargetName + "!");
627         }
628     }
629
630     UpdateUi();
631 }
632
633 private void EndBattle(bool playerWon)
634 {
635     SetBattleButtons(false);
636     UpdateUi();
637
638     if (playerWon)
639     {
640         if (bossBattle)
```

```
637         {
638             AddVictoryLog("The dragon crashes to the ground. You
win!");
639             AddVictoryLog("You defeated every enemy and slayed
the Ancient Dragon!");
640             statusLabel.Text = "Victory. Press Restart to begin a
new run.";
641             gameOver = true;
642         }
643     else
644     {
645         AddVictoryLog("You defeated the " + currentEnemy.Name
+ ".");
646
647         if (currentLevel <= 6)
648         {
649             waitingForUpgrade = true;
650             SetUpgradeButtons(true);
651
652             if (currentLevel == 6)
653             {
654                 statusLabel.Text = "Choose one final upgrade
before the boss.";
655                 AddLog("");
656                 AddWarningLog("You survived the 6 levels.
Choose one final upgrade before the Ancient
Dragon.");
657             }
658             else
659             {
660                 statusLabel.Text = "Choose one upgrade to
continue.";
661                 AddLog("");
662                 AddWarningLog("Choose an upgrade.");
663             }
664         }
665     }
666 }
667 else
668 {
669     if (bossBattle)
670     {
671         AddDefeatLog("The dragon burns through your final
defense.");
672     }
673     else
674     {
675         AddDefeatLog("The " + currentEnemy.Name + " defeated
you.");
```

```
676         }
677
678         AddDefeatLog("Your journey ends in the Dark Ages.");
679         statusLabel.Text = "Defeat. Press Restart to try again.";
680         gameOver = true;
681         SetUpgradeButtons(false);
682     }
683 }
684
685 private void UpgradeHealthButton_Click(object sender, EventArgs e)
686 {
687     if (!waitingForUpgrade || player == null)
688     {
689         return;
690     }
691
692     player.MaxHealth += 10;
693     player.Health = player.MaxHealth;
694     AddInfoLog("Your max health is now " + player.MaxHealth + " ");
695     FinishUpgrade();
696 }
697
698 private void UpgradeDamageButton_Click(object sender, EventArgs e)
699 {
700     if (!waitingForUpgrade || player == null)
701     {
702         return;
703     }
704
705     player.BonusDamage += 2;
706     player.Health = player.MaxHealth;
707     AddInfoLog("Your bonus damage is now " + player.BonusDamage + " ");
708     FinishUpgrade();
709 }
710
711 private void UpgradePotionButton_Click(object sender, EventArgs e)
712 {
713     if (!waitingForUpgrade || player == null)
714     {
715         return;
716     }
717
718     player.Potions += 1;
719     player.Health = player.MaxHealth;
720     AddInfoLog("You now have " + player.Potions + " potion(s).");
721     FinishUpgrade();
722 }
```



```
723
724     private void FinishUpgrade()
725     {
726         waitingForUpgrade = false;
727         SetUpgradeButtons(false);
728         currentLevel++;
729         StartCurrentLevel();
730     }
731
732     private void SetBattleButtons(bool enabled)
733     {
734         attackButton.Enabled = enabled;
735         blockButton.Enabled = enabled;
736         healButton.Enabled = enabled;
737     }
738
739     private void SetUpgradeButtons(bool enabled)
740     {
741         upgradeHealthButton.Enabled = enabled;
742         upgradeDamageButton.Enabled = enabled;
743         upgradePotionButton.Enabled = enabled;
744     }
745
746     private void UpdateUi()
747     {
748         if (player == null)
749         {
750             return;
751         }
752
753         levelLabel.Text = bossBattle ? "Level: Final Boss" : "
            Level: " + currentLevel;
754         classLabel.Text = "Class: " + player.ClassName;
755         playerHealthLabel.Text = "Health: " + player.Health + "/" +
            player.MaxHealth;
756         weaponLabel.Text = "Weapon: " + player.Weapon.Name;
757         bonusDamageLabel.Text = "Bonus Damage: " + player.BonusDamage;
758         potionLabel.Text = "Potions: " + player.Potions;
759
760         playerHealthBar.Maximum = Math.Max(1, player.MaxHealth);
761         playerHealthBar.Value = Math.Max(0, Math.Min(player.Health,
            playerHealthBar.Maximum));
762
763         if (currentEnemy != null)
764         {
765             enemyNameLabel.Text = "Enemy: " + currentEnemy.Name + " "
                + currentEnemy.EnemyClassName + " (" +
                currentEnemy.Weapon.Name + ")";
766             enemyHealthLabel.Text = "Enemy Health: " +
```

```
currentEnemy.Health + "/" + currentEnemy.MaxHealth;
767
768     enemyHealthBar.Maximum = Math.Max(1,
769         enemyHealthBar.Value = Math.Max(0, Math.Min
            (currentEnemy.Health, enemyHealthBar.Maximum));
770     }
771     else
772     {
773         enemyNameLabel.Text = "Enemy: -";
774         enemyHealthLabel.Text = "Enemy Health: -";
775         enemyHealthBar.Value = 0;
776     }
777 }
778
779 private void AddLog(string text)
780 {
781     AddLog(text, Color.WhiteSmoke);
782 }
783
784 private void AddLog(string text, Color color)
785 {
786     logBox.SelectionStart = logBox.TextLength;
787     logBox.SelectionLength = 0;
788     logBox.SelectionColor = color;
789     logBox.AppendText(text + Environment.NewLine);
790     logBox.SelectionColor = logBox.ForeColor;
791     logBox.ScrollToCaret();
792 }
793
794 private void AddHeaderLog(string text)
795 {
796     AddLog(text, Color.Gold);
797 }
798
799 private void AddPlayerLog(string text)
800 {
801     AddLog(text, Color.LightGreen);
802 }
803
804 private void AddEnemyLog(string text)
805 {
806     AddLog(text, Color.Salmon);
807 }
808
809 private void AddInfoLog(string text)
810 {
811     AddLog(text, Color.LightSkyBlue);
812 }
```

```
813
814     private void AddWarningLog(string text)
815     {
816         AddLog(text, Color.Khaki);
817     }
818
819     private void AddVictoryLog(string text)
820     {
821         AddLog(text, Color.MediumSpringGreen);
822     }
823
824     private void AddDefeatLog(string text)
825     {
826         AddLog(text, Color.OrangeRed);
827     }
828
829     private BattleAction GetEnemyAction(Enemy enemy, bool isBoss)
830     {
831         if (enemy.Potions > 0 && enemy.Health <= enemy.MaxHealth / 3)
832         {
833             int healChance = random.Next(1, 101);
834
835             if (healChance <= 40)
836             {
837                 return BattleAction.Heal;
838             }
839         }
840
841         int blockChance;
842
843         if (isBoss)
844         {
845             blockChance = 18;
846         }
847         else
848         {
849             blockChance = 24;
850         }
851
852         if (enemy.Weapon.Type == WeaponType.Bow || enemy.Weapon.Type == WeaponType.Wand)
853         {
854             blockChance -= 8;
855         }
856
857         if (enemy.Health < enemy.MaxHealth / 2)
858         {
859             blockChance += 8;
860         }
```

```
861
862         int roll = random.Next(1, 101);
863
864         if (roll <= blockChance)
865         {
866             return BattleAction.Block;
867         }
868
869         return BattleAction.Attack;
870     }
871
872     private Enemy CreateEnemy(int level)
873     {
874         int classNumber = random.Next(1, 6);
875         string className = "";
876         Weapon weapon = null;
877
878         if (classNumber == 1)
879         {
880             className = "Knight";
881             weapon = new Weapon(WeaponType.Sword, "Sword", 5, 9);
882         }
883         else if (classNumber == 2)
884         {
885             className = "Vanguard";
886             weapon = new Weapon(WeaponType.BattleAxe, "Battle Axe", 6, 10);
887         }
888         else if (classNumber == 3)
889         {
890             className = "Hunter";
891             weapon = new Weapon(WeaponType.Bow, "Bow", 4, 10);
892         }
893         else if (classNumber == 4)
894         {
895             className = "Rogue";
896             weapon = new Weapon(WeaponType.Dagger, "Daggers", 4, 9);
897         }
898         else
899         {
900             className = "Mage";
901             weapon = new Weapon(WeaponType.Wand, "Wand", 5, 10);
902         }
903
904         string enemyName = enemyNames[random.Next(enemyNames.Length)];
905
906         while (enemyName.ToLower().Contains(className.ToLower()) ||
907             className.ToLower().Contains(enemyName.ToLower()))
908         {
909         }
```

```
908         enemyName = enemyNames[random.Next(enemyNames.Length)];
909     }
910
911     int health = 22 + (level * 4) + random.Next(0, 5);
912     Enemy enemy = new Enemy(enemyName, className, health, weapon);
913
914     if (level >= 4)
915     {
916         enemy.BonusDamage = 1;
917     }
918
919     if (level >= 6)
920     {
921         enemy.Potions = 1;
922     }
923
924     return enemy;
925 }
926 }
927 }
```